Introduction to Artificial Intelligence – Final Project Submission

Gilad Mizrahi

March 2026

All related code and results can be found in the GitHub repository

1 Regression Task

We first analyze the data from the Auto MPG dataset. There are 6 rows with missing values on the `horsepower` feature, so we drop these rows. Next, we plot the distribution of our target variable `mpg`1 and its relationships between features2, as well as a correlation matrix3. We can already notice a strong relationship with `displacement`, `horsepower` and `weight`, as well as correlation between themselves. Therefore, in order to avoid high variance we choose to use all of the given features for prediction and also standardize all features to ensure uniformity (between features) and later for KNN.

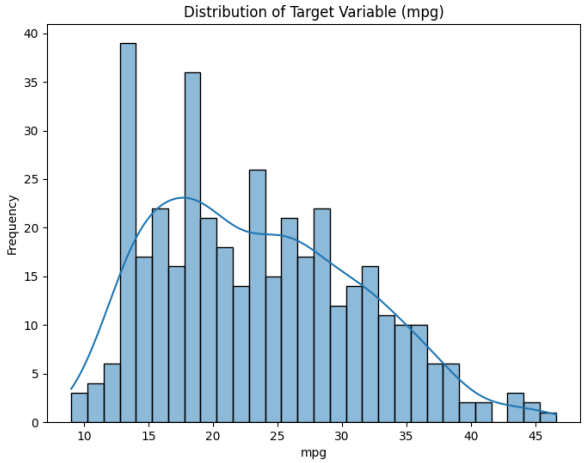


Figure 1: Distribution of mpg

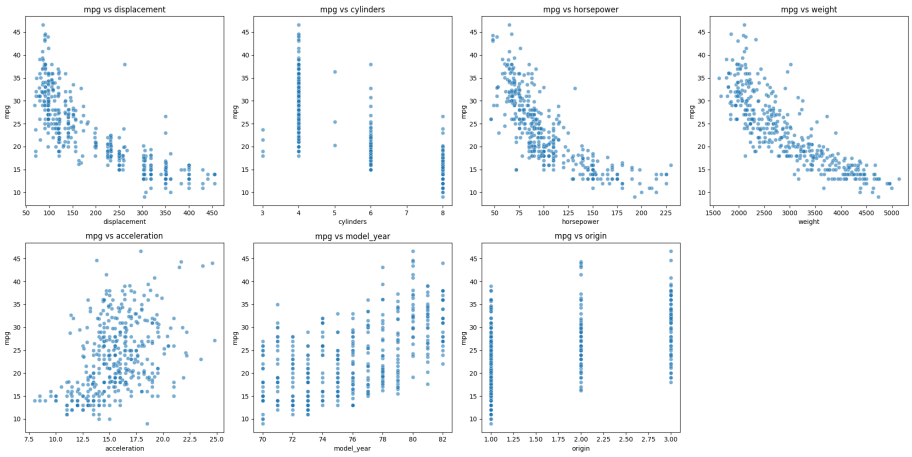


Figure 2: Relationship of mpg with individual features

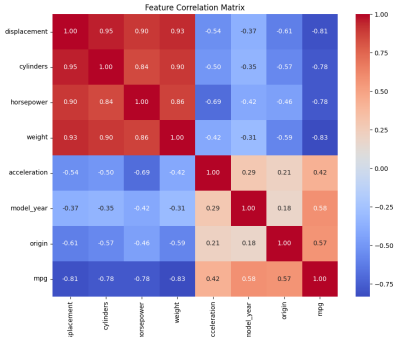


Figure 3: Feature correlation matrix

We begin with a simple linear regression model (dataset is split into 3:1:1 train-validate-test sets) and plot the prediction vs ground truth on the validation set4. We now evaluate the prediction using several metrics1. We observe from the plot that a linear assumption is reasonable, and some limitations such has a few outliers, as well as some low variance (which is actually good).

MSE	8.0820
RMSE	2.8429
MAE	2.1966
R^2	0.8329

Table 1: Linear regression performance

Moving on, we now try to use polynomial regression for degrees 1 to 5 (although

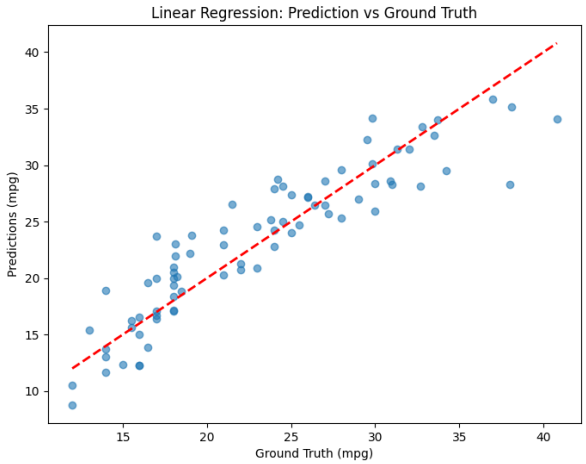


Figure 4: Linear regression prediction vs ground truth

1 is just linear), as we assume that by that point we will almost surely run into an overfitting issue. We plot the MSE for each degree5 and observe that by degree 4 we indeed run into overfitting: the training MSE is almost 0, but the validation MSE increased by a few orders of magnitude, meaning we get low bias and high variance as the complexity of the model increases. Overall, we choose the linear model (degree 1) due to retaining the lowest validation error1.

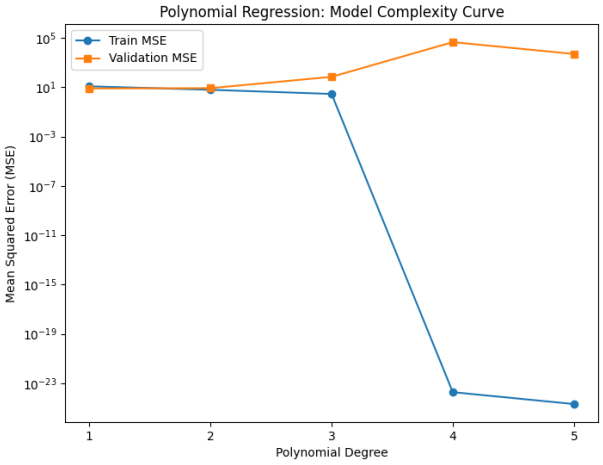


Figure 5: Polynomial regression complexity curve

For the next part, we use K-Nearest Neighbors (KNN) to analyze a non-parametric regression model. Plotting the training and validation performance (MSE) for each $k \in \{1, \ldots, 50\}$6, we observe that the best number of groups based on validation performance is $k = 39$, so we plot its prediction vs ground truth7. In our case, KNN does not outperform (or just barely) the parametric models. This is because it truly shines when we have highly non-linear data relations and large amounts of data, both not present here.

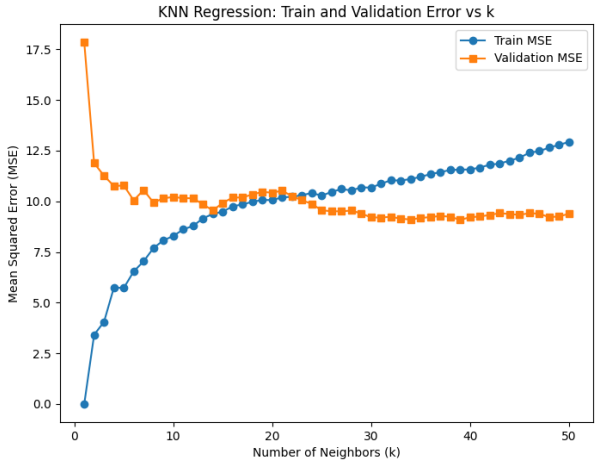


Figure 6: Training and validation performance for $k \in \{1, \ldots, 50\}$

Finally, we train a linear regression using batch GD and mini-batch GD (32 samples), comparing their losses with each epoch8. We immediately mini-batch GD having a lower loss with much fewer epochs. This is because despite not being deterministic and smooth like batch GD, we get many updates every epoch (instead of 1), and achieve faster loss optimization even if noisy.
We conclude our results and analysis by first considering the three models we have looked at: linear, polynomial, and KNN.
Linear models were the most simple and straightforward, no choice of parameters and generally constrained by data given; it can be used as a good reference for comparing other models due to their predictability.
Polynomial models are flexible and powerful, especially on non-linear data. However, they come with the cost of potential overfitting and high variance which we saw, so the polynomial degree should be chosen carefully; in our case, degrees of 2 and over did not yield a better MSE score on the validation set.
KNN are non-parametric, and as stated before shine with large data with low dimensionality (less features). Their biggest consideration is choosing the amount of neighbors (k), as we saw that our best performance was with a seemingly arbitrary $k = 39$. Higher and lower values result in high bias and high variance respectively.

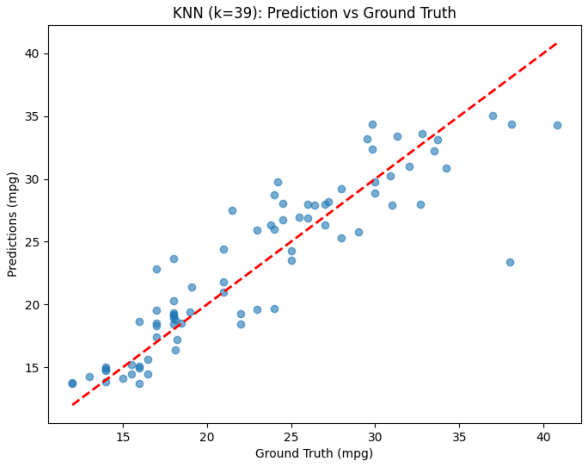


Figure 7: Prediction vs ground truth for KNN with $k = 39$

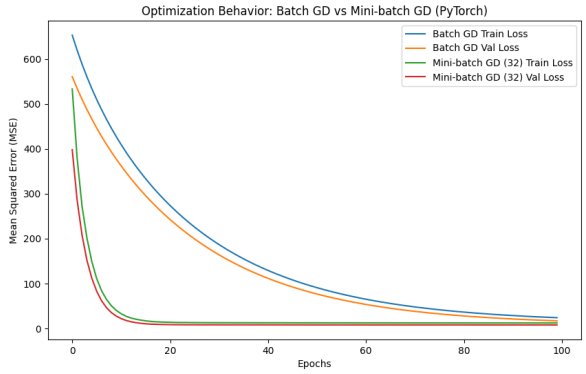


Figure 8: Regression Optimization: Batch GD vs Mini-batch GD

In conclusion, due to parameter consideration and overall best performance on the validation set, We would choose the **linear model**. We report the performance on the test set2.

MSE	10.6100
RMSE	3.2573
MAE	2.4052
R^2	0.7921

Table 2: Linear regression test performance

2 Classification Task

For this classical multi-class classification problem on the CIFAR-10 dataset, we use the following and show their respective suitable hyperparameter:

- Multiclass Logistic Regression (Softmax): regularization strength, which is inverse proportional to C
- Linear Support Vector Machine (SVM): Same as above
- K-Nearest Neighbors (KNN): Number of neighbors k

We take a global training an validation set with a ratio of 3:1[1].
For Softmax, we train and predict using $C \in \{10^{-5}, \ldots, 10^{-1}\}$ and plot the accuracy scores9. $C = 10^{-3}$ gives us the highest accuracy at 0.3520, so we choose that and plot its confusion matrix10

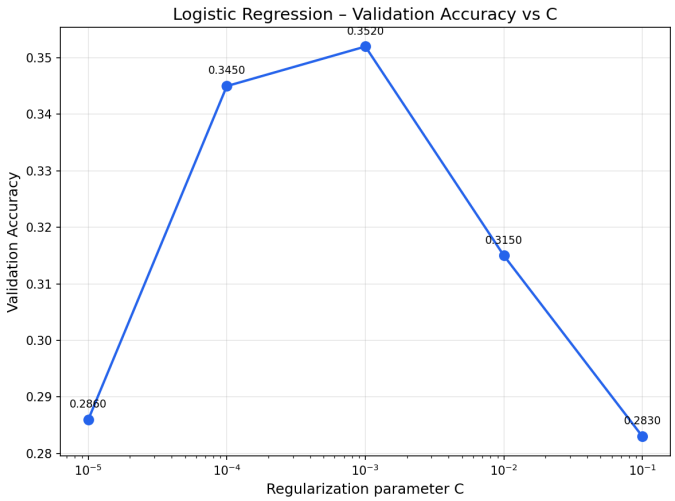


Figure 9: Softmax accuracy for each C

We repeat a similar process for SVM with $C \in \{10^{-6}, \ldots, 10^{-2}\}$, plotting each accuracy11. We get the best accuracy for $10^{-}4$ with 0.3590, plotting its confusion matrix12.
For the KNN model, we take a range of neighbors $k \in \{1, 3, \ldots, 49\}$ and plot each model's accuracy13. The highest accuracy comes at $k = 31$ with 0.27; we plot the corresponding confusion matrix14.
As discussed in the first part, KNN works best at low dimensionality. This is of course a problem when considering images, even with low resolution (here

[1]Due to very slow runtime, we had to cut the dataset size into 4000 samples (3000 train, 1000 validate). This may have affected the general accuracy, but should be fine for comparison between models

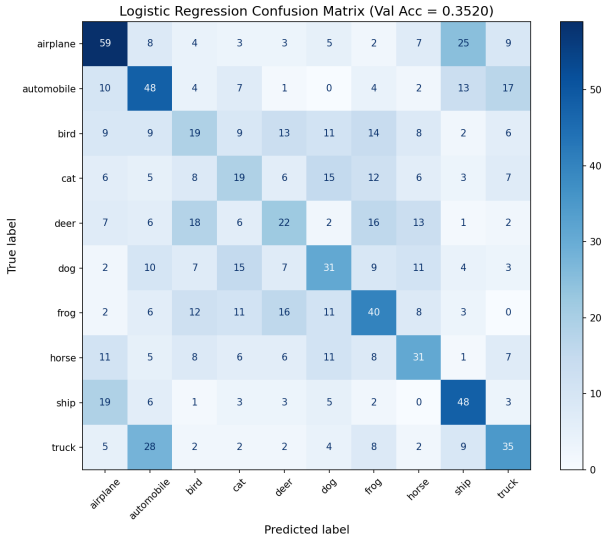


Figure 10: Softmax confusion matrix for $C = 10^{-3}$

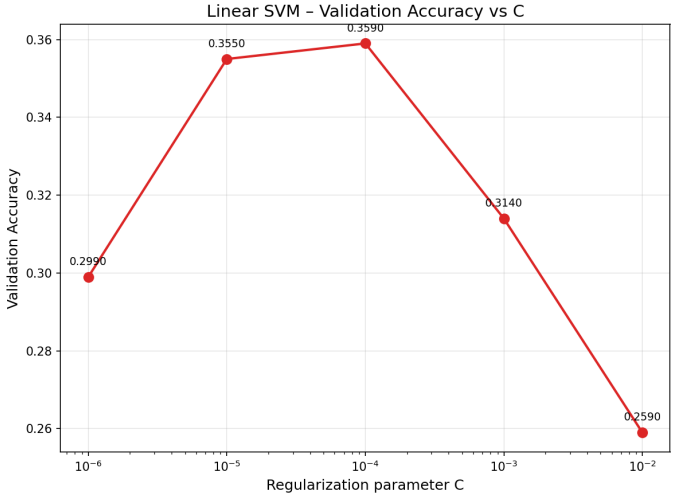


Figure 11: SVM accuracy for each C

we have 3072 dimensions). Logistic regression and linear SVM are both linear models and as such will come to a hard limit on their accuracy for highly non-linear data like this.

Overall, classical machine learning models struggle with images due to identity as raw, often flattened pixels: even a minor shift or rotation can dramatically change the features, which these models cannot learn how to handle properly

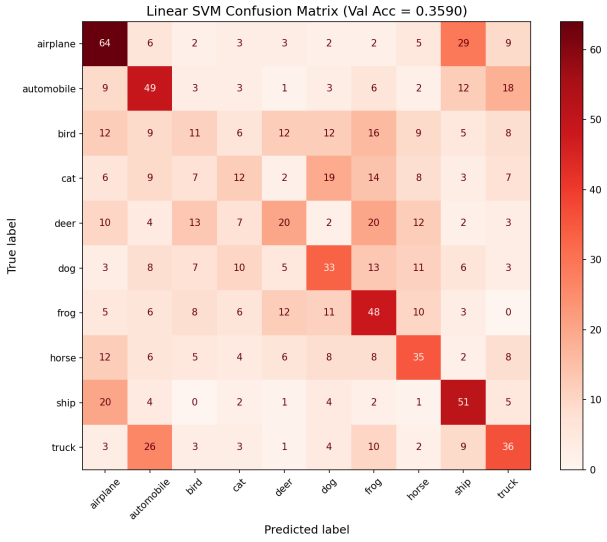


Figure 12: SVM confusion matrix for $C = 10^{-4}$

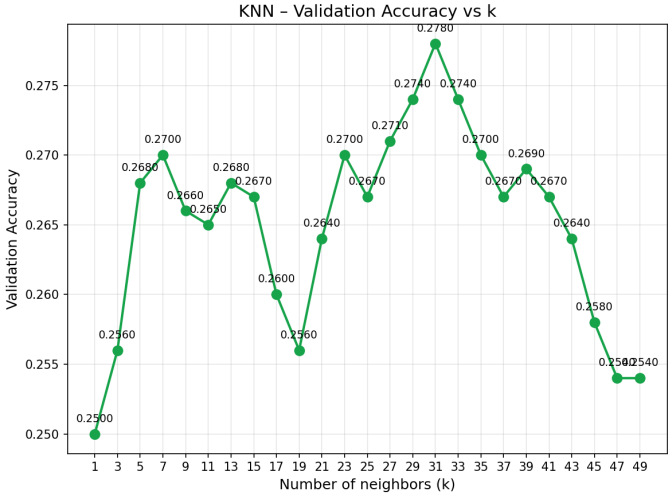


Figure 13: KNN accuracy for each k

like CNNs or similar models.
Our most accurate model was **Linear SVM** with an accuracy of 0.359. Its test accuracy comes at 0.3615.

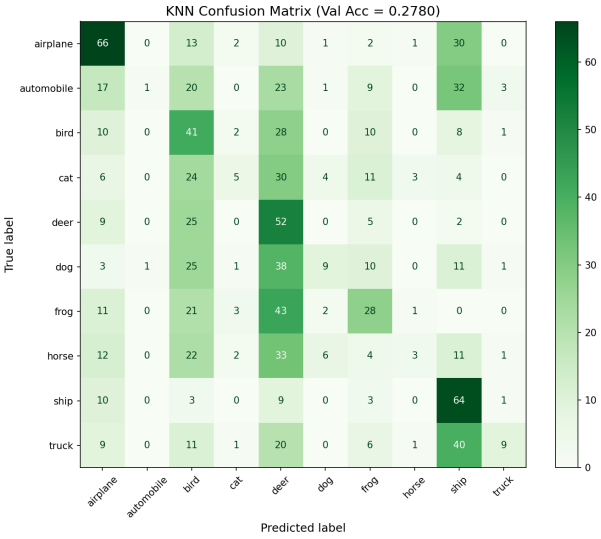


Figure 14: KNN confusion matrix for $k = 31$

3 PyTorch Task

We are instructed to find and implement a good classification model for the CIFAR-10. We choose the following permutation of models and their hyperparameters for an overall of 72 models, and compare their validation set accuracy over 5 epochs3. Implementing them as `nn.Module` classes and passing the data to `nn.Sequential` for convenience.

Model	Channels/Hidden layers	Optimizer	Weight decay	Dropout
FC	256, 128	SGD, Adam	$0, 10^{-4}$	$0, 0.25$
CNN	32, 64, 128 and 64, 128, 256	SGD, Adam	$0, 10^{-4}$	$0, 0.25$

Table 3: Comparison of different models and parameters

We conclude that the best model is a CNN (convolutional neural network) with Adam with learning rate of 10^{-3}, dropout applied. We also learn from the comparisons that CNN models are much superior to FC (fully connected) models when handling large data with high dimensionality. Also, the Adam optimizer slightly beat SGD at every turn, making it the preferred choice.
Training our selected model for 10 epochs, we get a final score of 86.68% on the test dataset. This is miles ahead of our classical classification, which reached only 36.15%.

4 PyTorch Regression Task

We adapt the previous classification problem into a regression one, with the Auto MPG dataset. Here, we also run a similar permutation of different models: FC with 3 different hidden layers: $(64, 32), (128, 64), (256, 128, 64)$, Adam and SGD, learning rates of $10^{-4}, 10^{-3}, 10^{-2}$, dropout of 0.2 or none and weight decay of 10^{-4} or none.
The best one was FC-256-128-64 with SGD, learning rate of 10^{-2}, weight decay and dropout applied[2]. We plot the MSE loss for the training and validation sets for the 200 epochs15 and the subsequent prediction vs ground truth16

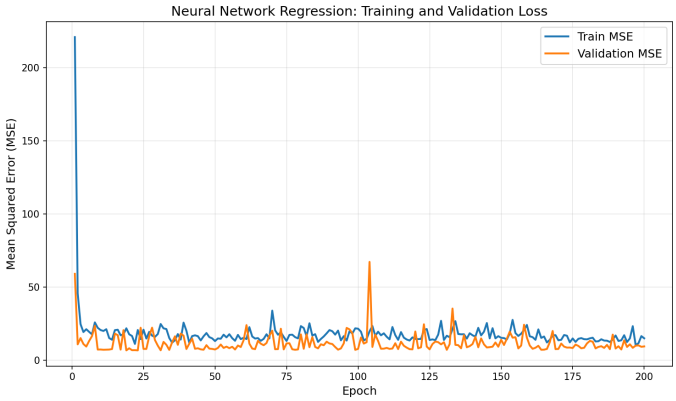


Figure 15: MSE loss for training and validation sets using FC-256-128-64, SGD, wd = 0.01, dropout = 0.2

We now compare our new model with the previous ones from the first task4. There is a noticeable improvement over the simple regression models we first tested.

Model	**Test MSE**	**Test RMSE**	**Test R^2**
Linear Regression	10.6100	3.2573	0.7921
Poly Regression (deg=1)	10.6100	3.2573	0.7921
KNN (k=39)	10.4380	3.2308	0.7955
Neural Network (PyTorch)	6.5829	2.5657	0.8710

Table 4: Model Performance Metrics

[2]Due to automatic selection, we found out this might change depending on the environment the notebook is run on; overall, this results in an insignificant change of loss

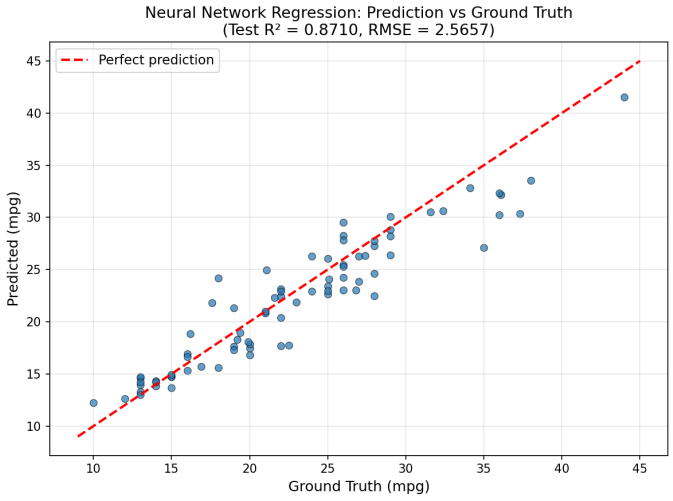


Figure 16: Prediction vs ground truth for FC-256-128-64, SGD, wd = 0.01, dropout = 0.2